

Forecasting Mobile Network Traffic in Milan: What Beats a Seasonal Naive Baseline?

Samuel Ihimbazwe
African Leadership University
s.ihimbazwe@alustudent.com

Code: <https://github.com/Samkwizera/Time-Series-Forecasting>

Abstract

Short-term forecasts of cellular traffic let operators pre-allocate capacity and power down idle carriers, but each cell follows the rhythm of the area it serves. We compare three forecasting families under one protocol on the 20.8 GB Telecom Italia Milan dataset (10 000 cells, 10-minute resolution, 62 days): seasonal ARIMA with exogenous regressors (SARIMAX), gradient-boosted trees (LightGBM), and recurrent networks (LSTM/GRU). A streaming ingestion pipeline reduces an estimated 16.7 GB naive load to a measured peak of 1,551 MB per ingestion day. Models are evaluated on 7 cells at 1, 6 and 24-hour horizons over a test window that deliberately contains Christmas and New Year. The main result is negative. The 24-hour seasonal naive is the strongest method overall (mean MASE 0.863), and no learned model beats it beyond one hour ahead. SARIMAX (1.427) and LightGBM (1.432) are statistically indistinguishable in aggregate, and the horizon separates the methods more sharply than the model family does: SARIMAX is the single best method in the study at $h=1$ (0.399), while LightGBM is the stronger of the two at $h=24$. The reason is structural. The baseline re-anchors on a recent observation, so it absorbs a sudden change in level without having to estimate it. How far a method must extrapolate matters more here than how much capacity it has.

Index Terms

mobile traffic forecasting, time series, SARIMA, gradient boosting, recurrent neural networks, forecast evaluation, Telecom Italia Big Data Challenge

I. INTRODUCTION

Cellular networks are dimensioned for peak demand, yet demand in any single cell swings by an order of magnitude over a day. If the load of the coming hours can be anticipated, an operator can pre-allocate spectrum, steer users between overlapping cells, or switch off carriers overnight to save energy. The costs of error are asymmetric: under-forecasting the evening peak drops sessions, while over-forecasting at 4 a.m. wastes power. The problem is also heterogeneous. A cell covering an office block, a residential street or a nightlife district has a different daily shape, so a single citywide model may not serve all of them equally.

The Telecom Italia Big Data Challenge dataset [1], [2] makes the question tractable. It records SMS, call and Internet activity on a 100×100 grid over Milan, at 10-minute resolution, for two months. At 20.8 GB it is also big enough that the first engineering decision is part of the problem: how to load it at all inside the 12–30 GB of RAM a free Colab or Kaggle session gives you.

Objective. To build a reproducible, memory-bounded pipeline for hourly Internet-activity forecasting on this dataset, and to compare three model families with different inductive biases under one common evaluation protocol.

Research question. *How does forecasting accuracy differ between a statistical model (SARIMAX), a gradient-boosted tree model (LightGBM) and a recurrent network (LSTM/GRU), across grid cells with different activity profiles and across horizons of 1, 6 and 24 hours, and do any of them beat a well-specified naive benchmark?*

Two aspects distinguish this study from prior work on the same data. Results are reported per cell and per horizon instead of as a single aggregate error, which is how the horizon effect becomes visible. And the benchmark is a properly seasonal naive instead of an untuned ARIMA, which changes the conclusion entirely. Cell labels used throughout are inferred from traffic-profile shape. They are not verified land-use categories, and we are careful not to argue as though they were.

II. RELATED WORK

Barlacchi et al. [2] document the dataset and note its strong daily and weekly periodicity and the heavy concentration of activity in the city centre. Our exploratory analysis confirms both quantitatively, and both shape our modelling choices. Furno et al. [3] show across ten cities that mobile-traffic “signatures”, the normalised weekly profiles of cells, cluster into a small number of recurring classes. This motivates our decision to let profile shape define cell types by clustering, instead of hand-picking landmarks and assuming their character, and then to ask whether the model ranking differs across those types.

Most forecasting work on the Milan grid uses deep spatio-temporal architectures. Wang et al. [4] combine autoencoders with LSTMs to capture inter-cell spatial dependence; Zhang et al. [5] treat the grid as an image and apply densely connected CNNs; Huang et al. [6] compare RNN, 3D-CNN and CNN-RNN variants. These studies report that deep models beat ARIMA. Two features of their protocols limit how much that comparison establishes: the statistical baseline is typically a non-seasonal or lightly tuned ARIMA, and errors are reported as one aggregate over the whole grid.

Two strands of the forecasting literature warn against reading such results as settled. Makridakis et al. [7] find that in controlled comparisons, well-specified statistical methods frequently match or beat machine-learning methods on short univariate series, and our series are short, at nine weeks. Januschowski et al. [8] explain why gradient-boosted trees have dominated recent forecasting competitions: they absorb calendar covariates and heterogeneous series with little tuning. Hyndman and Athanasopoulos [9] argue that any such comparison is meaningless without a naive benchmark and a significance test.

These cautions shaped the design directly. The statistical model is a properly seasonal SARIMAX instead of a straw-man ARIMA. The tree model receives exactly the same calendar information as the neural network, so the comparison tests inductive bias and not feature access. Every model is scored against seasonal-naive forecasts using scale-free MASE [10], and all differences are tested with Diebold–Mariano [11] using the Harvey–Leybourne–Newbold small-sample correction [12]. As Section VI shows, the benchmark that Makridakis et al. recommend is the one that ends up winning.

III. DATASET AND DATA PREPARATION

A. Data

The dataset [1] is 62 daily tab-separated files (1 Nov 2013 – 1 Jan 2014), 20.8 GB in total, with columns `square_id`, `time_interval` (Unix ms, UTC), `country_code`, and five activity measures (`sms_in`, `sms_out`, `call_in`, `call_out`, `internet`). Values are proportional to call-detail-record counts, rescaled by an undisclosed constant, so they only mean anything in relative terms. That is one reason every headline metric here is scale-free. Each (cell, interval) pair appears once per country code, so a single day holds roughly 4.5 million rows (308 MB).

B. Memory management

One day loaded with default `pandas` dtypes is estimated at 275 MB, and the whole period at about 16.7 GB. That is several times the RAM we have. Three decisions close the gap (Table I).

- 1) **Stream and aggregate before loading.** Each file is scanned lazily with the Polars streaming engine, with a chunked `pandas` path as a fallback, and rows are summed over `country_code` *during* the scan. The country dimension is irrelevant to load forecasting, and collapsing it removes roughly 90 % of all rows before any frame exists in memory. Of the three decisions this is the one that makes the pipeline feasible at all; the other two reduce a cost that is already manageable.
- 2) **Downcast at the point of creation.** Activities become `float32` and cell identifiers `uint16`. Timestamps are converted from UTC to Europe/Rome once, at ingestion, so that “8 a.m.” denotes the same thing in every downstream analysis and no later stage has to reason about time zones.
- 3) **Persist compact intermediates.** One `zstd`-compressed Parquet file per day is written, and from those a wide hourly matrix (`time` × 10 000 cells, `float32`) per activity. Only these matrices are read by the analysis and modelling stages. The 20.8 GB of raw TSVs is touched exactly once.

TABLE I: Memory footprint of the ingestion strategy (RSS = resident set size). The first two rows are estimates for a naive load; the rest are measured.

Stage	RSS (MB)	Time (s)
Naive pandas load, one day (est.)	275	–
Naive pandas load, all 62 days (est.)	17,058	–
Streaming aggregation, per day (median)	1,420	4.9
Streaming aggregation, all days (max, total)	1,551	300
wide hourly internet	1,514	17.5
citywide 10min	5,244	9.5
eda	567	10.9
tss	554	56.3

Peak resident set size during the per-day ingestion loop was 1,551 MB, and the 62 days took 5.0 minutes. One measurement runs against our own design. The later citywide 10-minute aggregation reached 5,244 MB, the highest point of the whole pipeline, because that stage builds a fine-grained frame that early aggregation does not protect. It fits the Kaggle environment, but it shows that the aggregate-early principle was not applied uniformly, and it is the stage we would restructure first. Every stage records peak RSS and wall time to `reports/tables/memory_log.csv`. The same instrumentation supplies the compute-cost comparison in Section VI, so accuracy and cost are measured by one mechanism.

C. Preprocessing

Missing (cell, interval) pairs mean zero activity, so they are filled with 0. The daily files are cut at UTC midnight while the analysis runs in local time, so the same local hour can appear in two files. The hourly matrix is therefore re-aggregated by timestamp after concatenation; plain concatenation would silently split those hours. Any residual gap in the hourly index is linearly interpolated. The forecasting target is hourly Internet activity, which accounts for 82 % of all recorded activity and is the quantity most relevant to capacity planning.

IV. EXPLORATORY ANALYSIS

This section reports only the analyses that carry a consequence for the forecasting design, and each subsection closes with that consequence.

A. Temporal structure

Fig. 1 shows citywide hourly totals. The dominant feature is a daily cycle modulated by a weekly one with lower weekend activity, and the final week of December departs from both. Fig. 2 decomposes the mean daily profile by day type. On weekdays Internet activity climbs steeply from 6 a.m., plateaus through office hours and peaks in the evening; weekends start later and lack the morning ramp; public holidays resemble Sundays. SMS and calls peak earlier and fall away in the evening while Internet activity is still high. The activities are driven by different behaviours and should not be pooled into a single target, which is why we model Internet activity alone. The hour-by-weekday heatmap (Fig. 3) makes the weekly rhythm explicit and shows Friday and Saturday nights extending later than other nights.

Consequence: day type predicts the shape of the day and not only its level, so every model is given calendar information about the *target* hour and not only about the origin.

B. Anomalous days

Daily totals scored with a robust z -score against the same weekday (Fig. 4) flag the days a model trained on ordinary weeks should find hardest. The flagged days coincide with public holidays: 1 November, Sant’Ambrogio (7 December, a Milan-only holiday), 8 December, and the Christmas period.

Consequence: a holiday indicator is informative and is supplied to every model, and a test window covering Christmas is a deliberate stress test, which we take into account when interpreting absolute error levels.

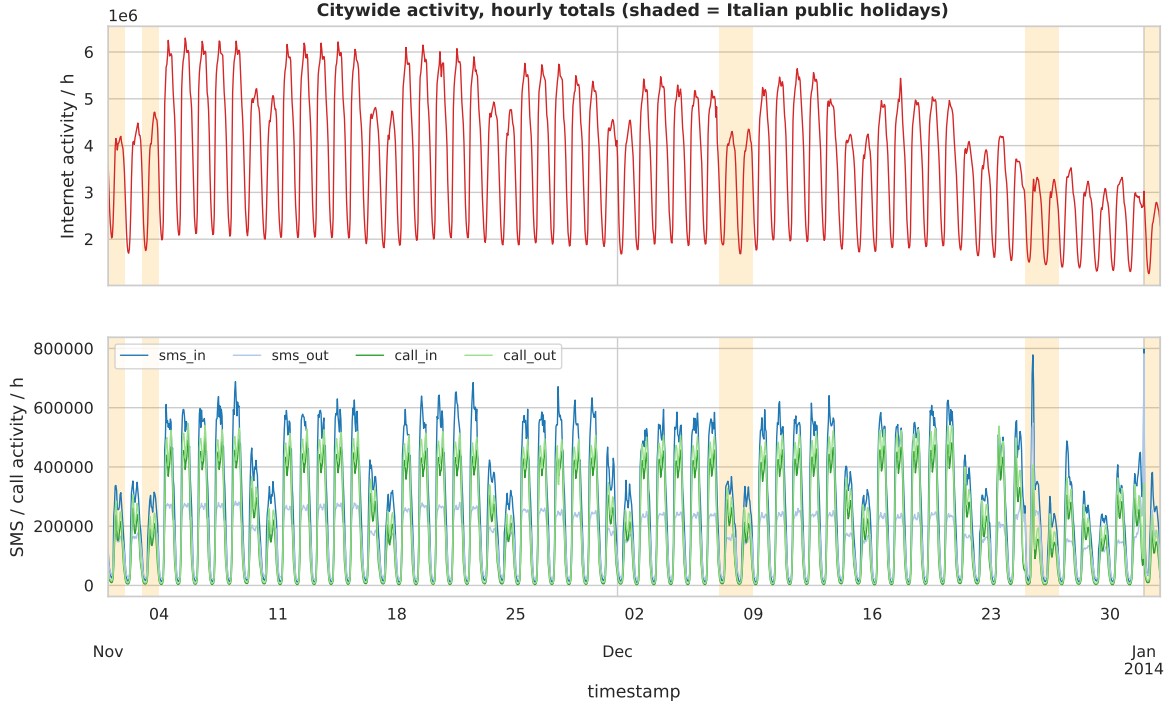


Fig. 1: Citywide hourly activity for all five measures. Shaded bands are Italian public holidays (1 Nov, 7–8 Dec, 25–26 Dec, 1 Jan).

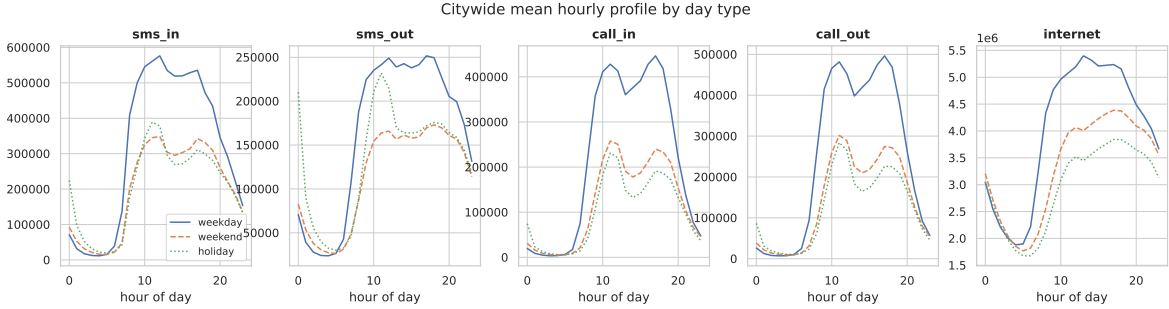


Fig. 2: Mean hourly profile by day type for each activity. Internet and voice peak at different hours, which is why they are not pooled.

C. Spatial structure and cell types

Activity is extremely concentrated. The busiest 1 % of cells carry 12 % of all Internet activity and the busiest 10 % carry 50 % (Gini 0.62). Fig. 5 shows why an aggregate evaluation would mislead: the centre dominates during office hours, by 9 p.m. the residential ring rises relative to the core, and outer cells are near-silent at every hour.

Following [3], each cell above the median activity level is represented by its normalised weekday 24-hour profile and clustered with k -means. The silhouette criterion favours $k = 2$ (0.47). We retain $k = 4$ (0.28) because the finer partition separates two business-shaped and two mixed/suburban-shaped profiles that we want to compare (Fig. 6). This is an interpretability choice made against the metric, and we flag it as such; $k = 4$ is not optimal under the silhouette score. Cluster names derive from centroid shape, using peak hour and the day/evening ratio, and are descriptive labels only.

For the forecasting experiments we select 7 cells: the landmarks used in the dataset’s companion material (Duomo, Bocconi, Navigli) plus the busiest cell of each cluster (duomo (#5060), bocconi (#4259), navigli (#4456), business

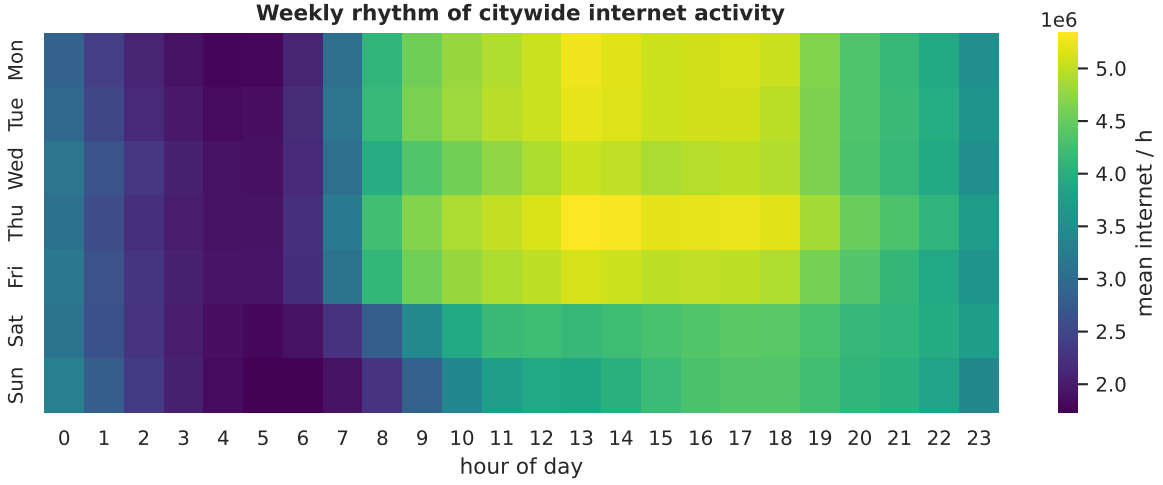


Fig. 3: Mean citywide Internet activity by hour of day and day of week.

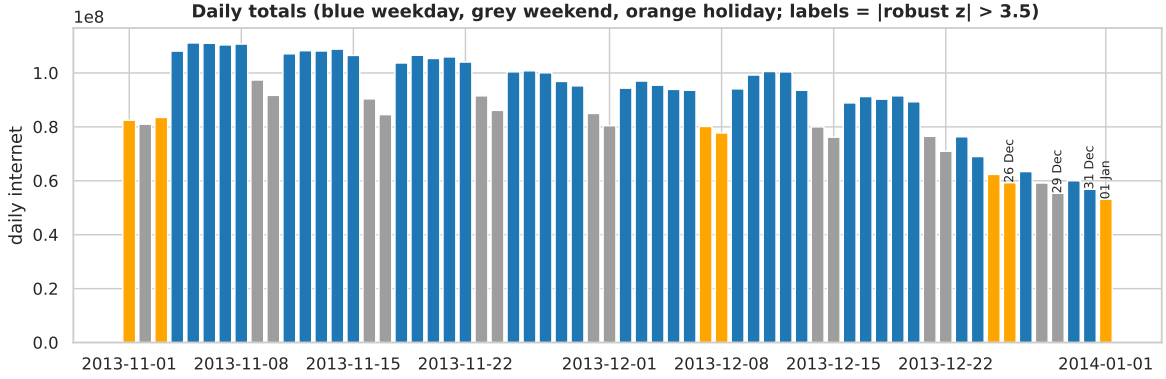


Fig. 4: Daily Internet totals; labelled bars have $|z| > 3.5$ relative to the same weekday. The flagged days are holidays.

(#4855), business-2 (#5161), mixed suburban (#5857), mixed suburban-2 (#4755)).

Consequence: evaluation is per cell, never on the citywide aggregate, whose smoothness flatters every model.

D. Stationarity, decomposition and autocorrelation

Table II reports ADF [13] and KPSS [14] tests. Using both matters here because they test opposing null hypotheses, and on the raw series they disagree for several cells. That disagreement is itself evidence of a trending seasonal level. The raw and log series are non-stationary in the KPSS sense. A seasonal difference at lag 24 makes them stationary under both tests, and a further first difference is not needed.

STL [15] with a 24-hour period attributes 81 % of the variance of the log series to the daily component and 6 % to the remainder. MSTL [16] with periods 24 and 168 assigns 81 % to the daily and 9 % to the weekly component (Fig. 7). The daily cycle is a first-order effect and the weekly cycle a clear second-order one. So we treat 24 as the seasonal period proper and handle 168 another way.

The ACF (Fig. 8) shows slowly decaying peaks at multiples of 24 and a pronounced peak at 168. The citywide values are instructive: $\rho(1) = 0.954$ and $\rho(24) = 0.927$ are both high, $\rho(168) = 0.794$ remains substantial, and $\rho(6) = 0.120$ is nearly zero. After seasonal differencing, the PACF has a negative spike at lag 24, the classic signature of a seasonal MA term, alongside significant low-order lags.

Consequences, which drive the entire Methodology section: the seasonal differencing orders are set to $d = 0$, $D = 1$; the variance that grows with the level motivates a $\log(1 + y)$ transform for all models; the lag set for the

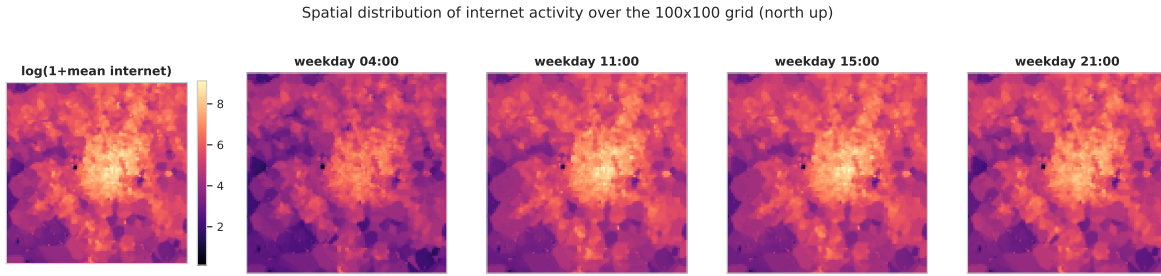


Fig. 5: Spatial distribution of Internet activity (log scale): weekday mean and snapshots at 04:00, 11:00, 15:00 and 21:00.

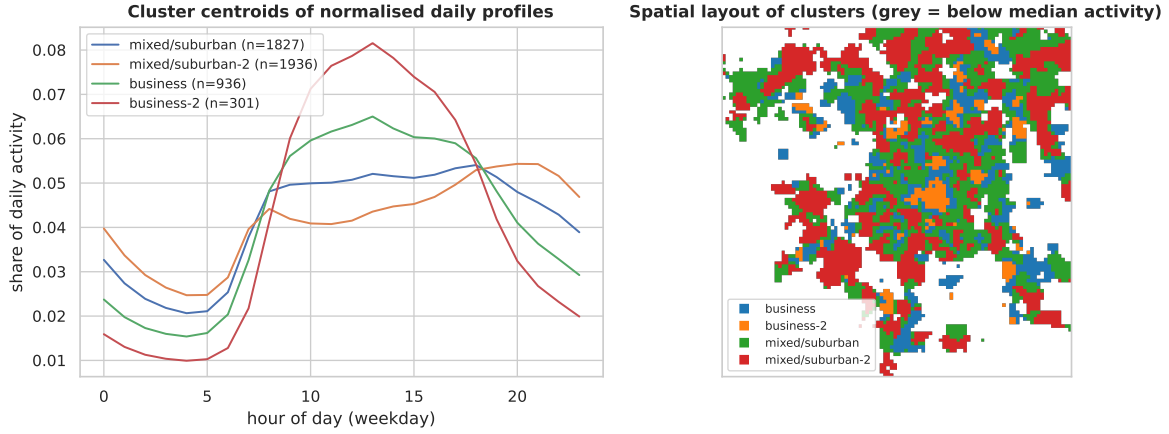


Fig. 6: Cluster centroids of normalised weekday profiles (left) and their spatial layout (right).

tree model is 1–24 plus daily steps to 168; one week is the candidate input window for the recurrent models; and the near-zero $\rho(6)$ predicts in advance that $h=6$ will be the horizon where persistence-style forecasts fail, which Section VI confirms.

V. METHODOLOGY

A. Forecasting setup

Given hourly Internet activity y_t for a cell, a forecast origin t , and a horizon $h \in \{1, 6, 24\}$, a model predicts $\hat{y}_{t+h}$ using data up to and including t . All models use the *direct* strategy, a separate mapping per horizon, so that errors at long horizons are not compounded by recursion and the three horizons remain independent measurements.

The split is chronological and identical for every model: training 1 Nov – 14 Dec, validation 15 – 21 Dec, test 22 Dec – 1 Jan. Forecasts are issued from every hour of the evaluation window (rolling origin), giving 241 forecasts per (model, cell, horizon).

Two properties of this design are enforced in code. First, an origin is admissible for a split only if *all* of its horizon targets fall inside that split, which prevents a forecast made near a boundary from reading across it. Second, the fitting window is defined per evaluation split: validation runs fit on training data only, test runs fit on training plus validation. Both rules live in a single function and are covered by a unit test asserting that no target of any admissible origin falls outside its split. Leakage enters time-series pipelines silently, so we treated this as a correctness property to be tested.

Placing the test window on Christmas and New Year is deliberate. Forecasting the holiday fortnight from six ordinary weeks is the situation an operator faces annually, and it is where models with different assumptions should diverge most. The cost of that choice is that validation contains no holiday, so validation cannot reward holiday handling, a limitation we return to in Section VI.

TABLE II: Stationarity tests on the citywide hourly Internet series.

Transform	ADF	p	KPSS	p	Verdict
raw	-2.14	0.230	3.63	0.010	non-stationary
log1p	-1.26	0.649	3.70	0.010	non-stationary
log1p, diff 1	-11.91	0.000	0.00	0.100	stationary
log1p, seasonal diff 24	-5.58	0.000	0.22	0.100	stationary
log1p, diff 1 + seasonal diff 24	-9.82	0.000	0.01	0.100	stationary

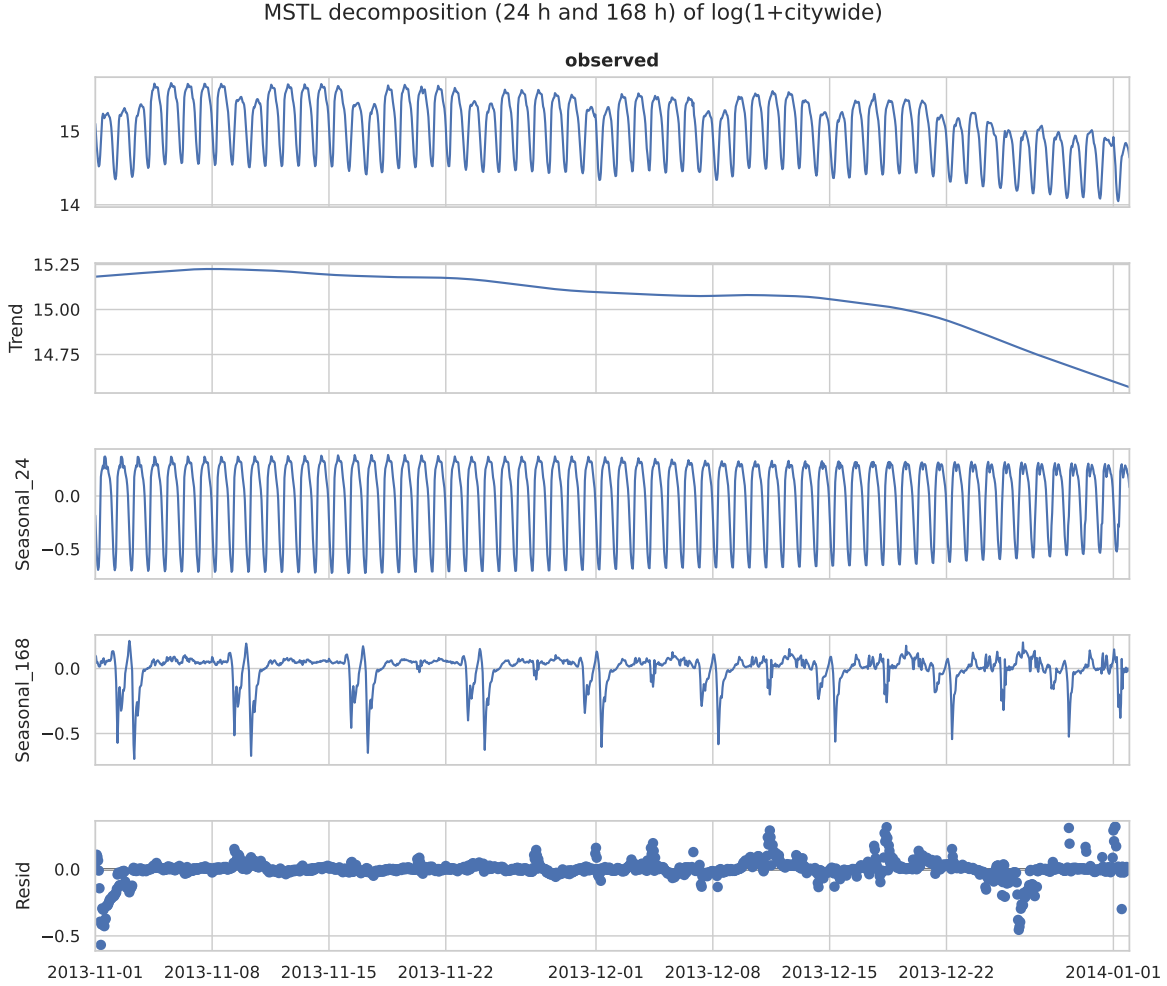


Fig. 7: MSTL decomposition of the log citywide series into trend, daily and weekly seasonal components, and remainder.

B. Input representation and preprocessing

All models operate on $z_t = \log(1 + y_t)$, standardised per cell, and predictions are inverted before scoring, so every reported metric is in original units. The standardisation statistics are computed on the training split alone, including for the final test runs whose fitting window is train plus validation. This is deliberate and conservative, since it keeps one fixed scale across all experiments, but it does mean the scale is centred on November and early December and does not absorb the validation week’s level decline.

Calendar information (hour, weekday, weekend and holiday flags, the last including Sant’Ambrogio) is supplied to every learned model in the representation each one can use. LightGBM receives them as columns for both the origin and the target hour; the recurrent models receive cyclical sine/cosine encodings at each input step plus the

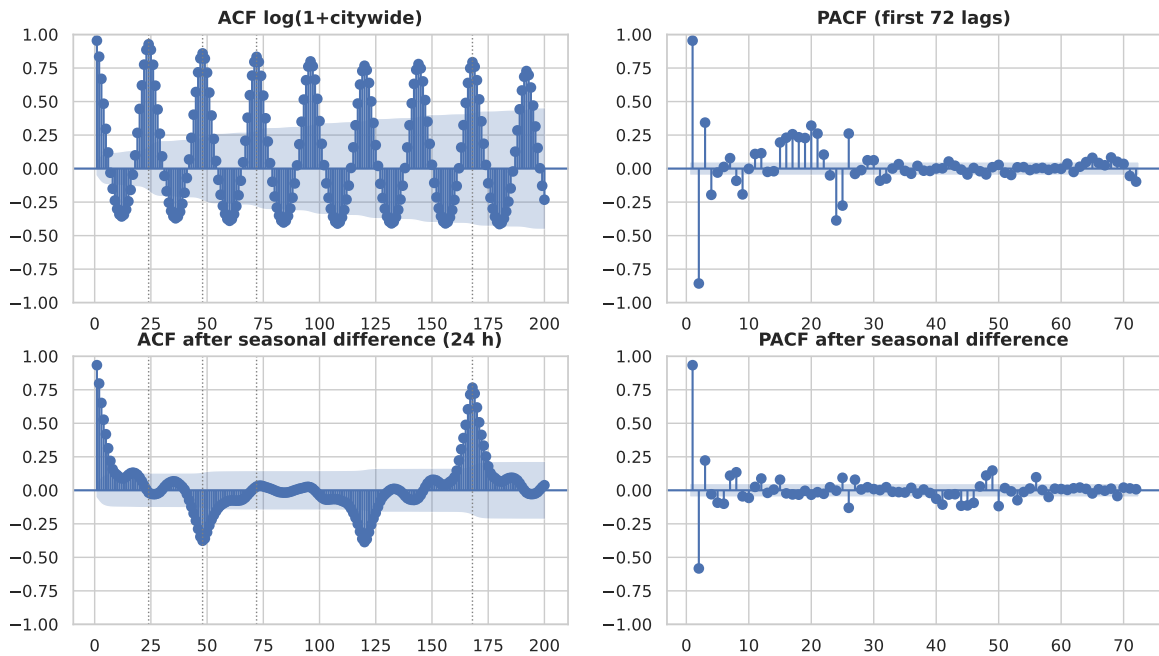


Fig. 8: ACF/PACF of the log citywide series before and after a 24-hour seasonal difference. Dotted lines at lags 24, 48, 72, 168.

encoding of every target hour. Giving both models the same information in their native form is what makes the comparison a test of inductive bias and not of feature engineering.

C. The three models and why they were chosen

The three families span different assumptions about what generates the series, which is what makes their relative performance interpretable.

Baselines. Naive ($\hat{y}_{t+h} = y_t$), 24-hour seasonal naive, and 168-hour seasonal naive. The 24-hour seasonal naive is also the scaling series for MASE. Following [9], these define the bar that any learned model must clear to have earned its complexity.

SARIMAX. A linear seasonal process: seasonal ARIMA $(p, 0, q)(P, 1, Q)_{24}$ on z_t , fitted per cell [17]. A seasonal period of 168 would make the state vector far too large, so the weekly cycle was offered instead as optional Fourier regressors, alongside an optional holiday dummy. Both were candidates for validation to accept or reject. Parameters are estimated once on the fitting window; the Kalman filter then runs over the evaluation window with fixed parameters, and h -step forecasts are read from each origin with `dynamic=True`, which makes them genuine multi-step forecasts and not filtered one-step updates. This mimics an operator who re-estimates periodically and forecasts continuously.

LightGBM. Nonlinear interactions over engineered lags: one regressor per horizon [18], trained jointly across the selected cells with a categorical cell identifier, so the trees share structure such as hour-of-day effects while still splitting on cell where profiles differ. Features are constructed strictly relative to the origin. The objective is ℓ_1 , which targets the conditional median, is robust to the heavy right tail of peak hours, and is the same loss that MAE and MASE measure, so training and evaluation are aligned.

Recurrent network. A learned representation of the recent sequence: LSTM [19] and GRU [20] cells in a shared PyTorch implementation [21]. The model reads 24 or 168 hours of $(z_t, \text{calendar}_t)$ and emits all three horizons from the final hidden state concatenated with a learned cell embedding and the calendar encoding of each target hour. Training uses Adam, ℓ_1 loss for the same reason as LightGBM, gradient clipping, a plateau learning-rate schedule, and early stopping with best-epoch restoration.

TABLE III: Experiment log (validation split). The rationale column is written *before* each run. The full log with all parameters is in `experiments/experiment_log.md`.

Model	Run	val MASE	val sMAPE	s	Rationale for the run
SARIMAX	s1	0.717	12.1	3	Minimal seasonal random walk + AR(1): the simplest structure consistent with the ACF (slow decay at multiples of 24). Reference point for what the seasonal difference alone buys.
SARIMAX	s2	0.659	10.8	44	Add MA terms at lag 1 and 24: PACF after seasonal differencing shows a negative spike at 24, the classic signature of a seasonal MA(1).
SARIMAX	s3	0.666	11.1	108	Weekly cycle cannot be modelled with a 168-period SARIMA (state too large). Add 3 Fourier pairs of period 168 h as exogenous regressors (dynamic harmonic regression).
SARIMAX	s4	0.669	11.2	139	Public holidays behave like Sundays; a holiday dummy lets the level drop without disturbing the seasonal state.
SARIMAX	s5	0.769	12.7	416	Check whether more harmonics (5) and a second AR lag improve or overfit; AIC vs validation MASE decides.
LightGBM	g1	0.642	10.7	7	Lags 1-24 only: how much can trees recover from the last day alone? Establishes the value of the seasonal/weekly lags added next.
LightGBM	g2	0.672	10.8	7	Add lags 36-168 (two-day and weekly cycle) identified in the ACF.
LightGBM	g3	0.604	9.7	10	Add calendar features of origin and target (hour, weekday, holiday) so the model can anticipate the day type of the target time, which lags alone cannot at 24 h.
LightGBM	g4	0.664	10.6	9	Add rolling means/stds (6, 24, 168 h) and differences: level and volatility summaries that reduce reliance on any single lag.
LightGBM	g5	0.682	10.9	6	Regularise: fewer leaves, lower learning rate and larger leaves to test whether g4 overfits the pooled data (compare train/val gap).
LightGBM	g6	0.652	10.4	9	Opposite direction: more capacity. Together with g5 this brackets the capacity sweet spot before the Optuna search.
LightGBM	Optuna (20 trials)	0.633	10.2	244	best trial opt05: num_leaves=67, learning_rate=0.0182, min_child_samples=32, feature_fraction=0.796, lambda_l2=0.00153
RNN	l1	0.721	11.5	12	Small network with one day of context, to see the floor of the architecture and whether training is stable.
RNN	l2	0.747	12.5	6	One week of context: the weekly cycle should now be visible to the network.
RNN	l3	0.714	11.5	6	Double the hidden size to test whether l2 was capacity-limited.
RNN	l4	0.793	12.6	7	Deeper network with dropout: more capacity, but regularised against the small number of training weeks.
RNN	l5	0.711	11.6	7	GRU has fewer parameters than LSTM at equal hidden size; test whether the gating simplification helps with 6 weeks of data.
RNN	l6	0.720	11.4	12	Lower learning rate and light weight decay to smooth the validation curve of the best previous configuration.

We exclude spatial CNNs, despite their prominence in [4], [5]. Judging neighbour information fairly would need a different input and sampling design, and mixing that change into the model-family comparison would make both harder to read.

D. Experimental design and tuning strategy

Tuning follows an iterative protocol. Each run is scored on validation, and each run’s parameters, score and the *reasoning for the next change* are appended to an experiment log before that next run is launched (Table III). The test split is touched exactly once per model, with the configuration chosen on validation.

SARIMAX and recurrent rounds are manual because each change tests a specific hypothesis drawn from Section IV. Run s2, for example, adds MA terms at lags 1 and 24 because the PACF after seasonal differencing shows the negative spike at 24 characteristic of a seasonal MA. For LightGBM, manual rounds first compare feature groups, after which 20 trials of TPE search [22] tune capacity and regularisation.

Three limitations of this design belong here, before the results, because they bound what the results can support. Early stopping uses the last seven days of the fitting window, so for the final test runs the early-stopping week is the same validation week that selected the configuration; the test split is never involved, but validation scores are correspondingly optimistic. All selection rests on a single seven-day window and a single seed, so differences of a few thousandths of MASE carry no evidential weight and we do not interpret them. And the Optuna space fixed `use_rolling=True` while varying only capacity and regularisation, so it searched a feature configuration the manual rounds had already found inferior.

TABLE IV: Test-set accuracy (22 Dec – 1 Jan), mean over the selected cells. $\text{MASE} < 1$ beats the 24-hour seasonal naive on the training scale.

Model	$h = 1 \text{ h}$		$h = 6 \text{ h}$		$h = 24 \text{ h}$		All horizons		
	MASE	sMAPE	MASE	sMAPE	MASE	sMAPE	MASE	sMAPE	MAE
Naive	0.471	13.3	2.094	54.5	0.873	24.2	1.146	30.7	1,068.9
Seasonal naive (24 h)	0.854	22.3	0.863	23.0	0.873	24.2	0.863	23.2	768.8
Seasonal naive (168 h)	2.380	51.0	2.369	51.0	2.326	51.5	2.358	51.2	1,816.0
SARIMAX	0.399	11.0	1.542	35.9	2.341	54.5	1.427	33.8	1,058.3
LightGBM	0.949	26.1	1.780	43.0	1.568	40.6	1.432	36.6	1,087.2
GRU	1.012	26.8	2.699	60.0	2.957	62.9	2.223	49.9	1,610.5

E. Evaluation

Errors are reported in original units as MAE, RMSE, sMAPE and MASE [10], per cell and horizon. MASE is the headline metric because the cells differ in volume by more than an order of magnitude, which makes unscaled errors incomparable across them. A MASE below 1 means a method beats the 24-hour seasonal naive *on the training scale*. Differences in absolute error are tested with Diebold–Mariano [11] using the Harvey–Leybourne–Newbold small-sample correction and $h - 1$ Newey–West lags [12]. Training time and peak memory are recorded per model so that accuracy can be weighed against cost.

VI. RESULTS AND DISCUSSION

A. Tuning trail

For SARIMAX, adding MA terms at lags 1 and 24 (s2) cut validation MASE from 0.717 to 0.659. The weekly Fourier regressors (s3, 0.666), the holiday dummy (s4, 0.669) and a larger specification (s5, 0.769) all failed to improve on that, so s2 was selected. The Fourier terms are the interesting failure, because they were motivated by a real feature of the ACF ($\rho(168) = 0.794$) and still did not help, which suggests the weekly cycle is already captured through the daily seasonal structure and recent lags.

For LightGBM, adding target-time calendar features produced the best manual run (g3, 0.604), confirming the Section IV expectation that a model must know the day type of the hour it is predicting. Rolling statistics worsened validation MASE to 0.664. The best of 20 Optuna trials reached 0.633, which also failed to beat g3, though the search held `use_rolling=True` fixed. What that shows is that no capacity setting rescues the rolling-feature variant. It does not show that g3 is a tuned optimum.

In the recurrent search, the 168-hour LSTM did not improve on the 24-hour version (0.747 versus 0.721); a larger LSTM recovered to 0.714 and the GRU variant scored 0.711. The last two differ by 0.003 MASE on a single seed, which is smaller than the run-to-run variation expected from random initialisation. The GRU was selected on the tie-break that it has fewer parameters at equal width, and not on a demonstrated accuracy advantage.

Several changes motivated by sound time-series reasoning therefore failed to generalise to the validation week. We report them because more structure without improvement is the first hint of the result that follows.

B. Comparative performance

Table IV gives test accuracy averaged over cells, Table V the cell-level breakdown, Table VI the significance counts, Table VII the compute cost, and Fig. 9 the headline comparison visually.

The 24-hour seasonal naive is the strongest method overall, at 0.863. It beats every learned model at 6 and 24 hours and stays below 1 at each horizon. The Diebold–Mariano counts agree: against SN-24, SARIMAX is significantly better in 7 of 21 cell-horizon comparisons and worse in 10; LightGBM is better in 2 and worse in 14; the GRU is better in 2 and worse in 15.

SARIMAX (1.427) and LightGBM (1.432) differ by 0.005 MASE, far less than the spread across cells and horizons in Table V. With one seed and seven cells we treat them as indistinguishable in aggregate and decline to rank them. The GRU is clearly weakest at 2.223, and that gap is large enough to be meaningful.

TABLE V: Test MASE by cell and horizon (best in bold; SN-24 and SN-168 are the 24-hour and 168-hour seasonal naives).

Cell	h	Naive	SN-24	SN-168	SARIMAX	LightGBM	GRU
bocconi	1	0.092	0.260	1.207	0.105	0.483	0.497
bocconi	6	0.325	0.271	1.212	0.516	1.084	1.803
bocconi	24	0.255	0.255	1.203	1.119	0.805	2.337
business	1	0.205	0.462	2.438	0.179	0.970	0.967
business	6	0.829	0.460	2.434	0.905	1.770	2.816
business	24	0.426	0.426	2.401	2.136	1.570	3.367
business-2	1	0.835	1.361	1.930	0.412	0.631	0.607
business-2	6	4.196	1.365	1.928	1.243	1.402	1.662
business-2	24	1.464	1.464	1.876	1.646	1.329	1.722
duomo	1	0.737	1.378	2.008	0.410	0.764	0.665
duomo	6	3.534	1.424	1.977	1.323	1.447	1.731
duomo	24	1.568	1.568	2.018	1.712	1.523	1.710
mixed suburban	1	0.539	0.896	2.857	0.487	0.925	1.068
mixed suburban	6	2.424	0.898	2.855	1.966	1.959	2.693
mixed suburban	24	0.927	0.927	2.878	3.202	1.654	3.384
mixed suburban-2	1	0.457	0.901	3.482	0.634	1.524	1.543
mixed suburban-2	6	1.661	0.875	3.440	2.558	2.391	3.840
mixed suburban-2	24	0.765	0.765	3.287	3.508	2.178	4.039
navigli	1	0.430	0.720	2.739	0.564	1.343	1.738
navigli	6	1.692	0.746	2.740	2.280	2.408	4.352
navigli	24	0.706	0.706	2.619	3.062	1.918	4.137

TABLE VI: Diebold–Mariano outcomes against the 24-hour seasonal naive ($p < 0.05$), over 21 cell-horizon comparisons per model. At $h = 24$ the naive and SN-24 forecasts coincide by construction ($y_{t+24-24} = y_t$), so their loss differential is identically zero and the test is undefined; those seven comparisons are excluded and not counted as ties.

Model	Sig. better	Sig. worse	Comparisons
GRU	2	15	21
LightGBM	2	14	21
SARIMAX	7	10	21

The horizon separates the methods more sharply than the model family does. SARIMAX is the single best method in the study at one hour (0.399, against 0.854 for SN-24 and 0.949 for LightGBM), then degrades steeply to 2.341 at 24 hours. LightGBM degrades far less, reaching 1.568 at $h=24$ and overtaking SARIMAX there. Across the cell-level table the ranking of learned models shifts with horizon far more systematically than it shifts with the cell label.

On the cell-type half of the research question the answer is negative. The results do not establish that activity profile determines which model wins: with seven cells and descriptive labels, the apparent variation across cells is not separable from noise, and we state that instead of reading a pattern into Table V.

C. Why the benchmark wins

The horizon pattern points to the mechanism. The 24-hour seasonal naive copies an observation from the same hour of the previous day, so the horizon changes only how stale that anchor is and never what is being copied, which is why its MASE is almost flat across horizons (0.854, 0.863, 0.873). A holiday fortnight is primarily a shift in *level* and not a change in daily shape, and re-anchoring absorbs a level shift automatically, one day late, without having estimated anything about it.

The learned models have the opposite profile. Each estimates a level from six weeks containing no comparable closure and then holds it, so accuracy decays with how far the model must extrapolate from the last observation. SARIMAX at $h=1$ is conditioned on the immediately preceding value and is the best method in the study; iterated

TABLE VII: Computational cost of the final test run per model, from the same instrumentation as Table I.

Model	Fit + forecast time (s)	Peak RSS (MB)
Baselines	0.1	1,048
GRU	12.0	1,710
LightGBM	9.1	1,058
SARIMAX	59.5	1,052

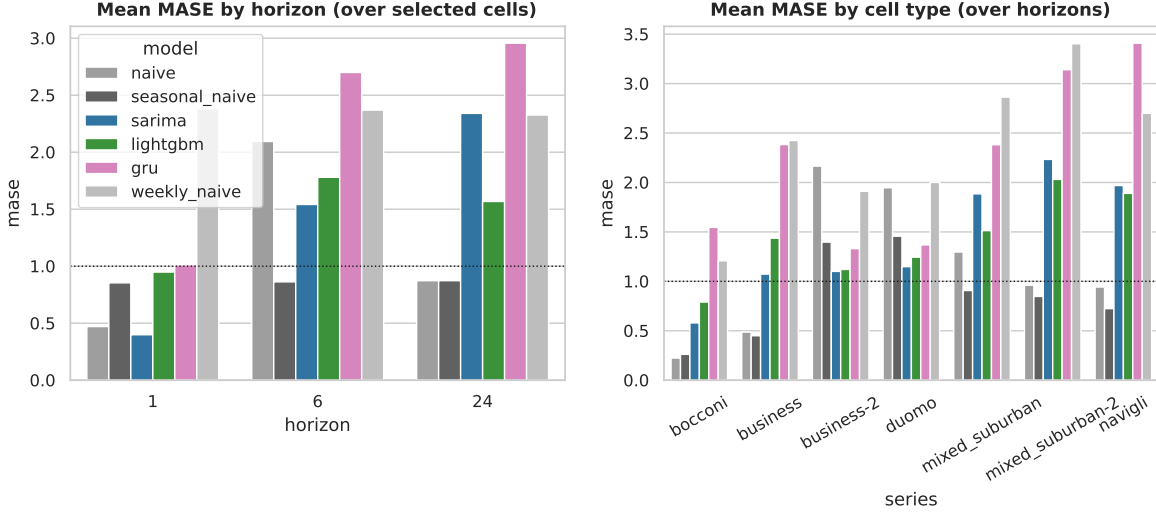


Fig. 9: Test MASE by horizon (left) and by cell (right). Dotted line marks seasonal-naive parity.

24 steps with `dynamic=True` it converges toward its fitted seasonal mean, a pre-holiday level, and degrades to 2.341. LightGBM degrades more gently at $h=24$ because its target-time calendar features let it condition on the day type of the hour being predicted instead of propagating its own forecasts.

The obvious explanation is incomplete, though. Mean residuals averaged over cells are +174 for SARIMAX, +559 for LightGBM and +540 for the GRU, so all three over-forecast during the holiday window (Fig. 11). But the 24-hour seasonal naive carries a mean bias of +208, larger than SARIMAX’s, while still achieving far better MASE at long horizons. So bias alone does not explain the outcome. What sets the benchmark apart is how steady its error stays as the horizon grows, and an account built on over-forecasting alone would miss that.

The $h=6$ column shows the same mechanism from the other side. The plain naive is excellent at $h=1$ (0.471) and collapses at $h=6$ (2.094), worse than any learned model, before recovering at $h=24$ (0.873, where it coincides with SN-24). This is what $\rho(6) = 0.120$ predicted in Section IV: six hours ahead, the last observation is nearly uncorrelated with the target, because six hours is roughly the distance from a daily trough to a daily peak. The horizon at which persistence fails was visible in the autocorrelation function before any model was fitted.

D. Failure analysis

Fig. 10 plots daily mean absolute error across the test period. Errors rise around Christmas and New Year for every method, but the peak day differs by model. LightGBM and the seasonal naive peak on 25 December, while SARIMAX and the GRU peak on 1 January. The two methods that extrapolate a fitted level are the two that fail latest, when the departure from the ordinary weekly pattern has persisted longest.

The most severe individual failures are 24-hour GRU forecasts for the Bocconi cell on 23 and 30 December, where the model predicts ordinary daytime levels against a large observed drop. Bocconi is a university cell, so its holiday closure is both deeper and earlier than the citywide average, a cell-specific effect whose magnitude six holiday-free training weeks cannot teach any model.

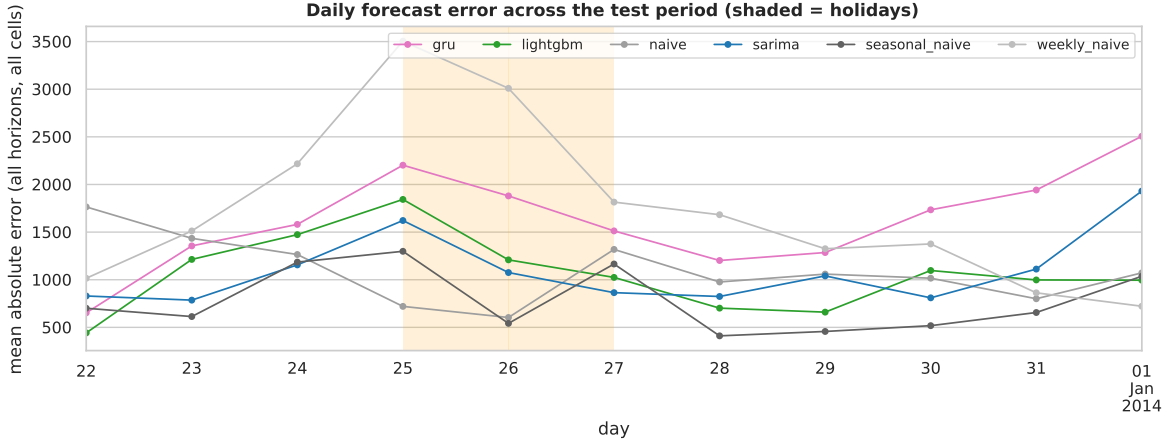


Fig. 10: Daily mean absolute error per model over the test period.

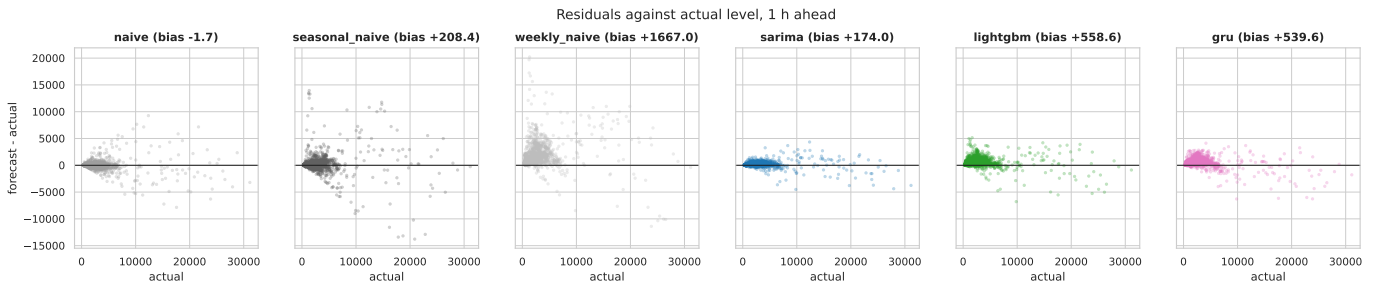


Fig. 11: One-hour-ahead residuals against the actual level. Positive means indicate over-forecasting.

We also report a fitting failure alongside the forecasting ones. One final SARIMAX fit, for the mixed-suburban cell, did not converge within the iteration limit, so that cell’s contribution to the SARIMAX aggregate should be treated with caution.

Together these point to a specific remedy. The models do not need more capacity, since the GRU has the most and performs worst. They need either a holiday regressor estimated across several seasons, or an explicit re-anchoring mechanism, such as differencing against the last observed same-hour value or online re-estimation, that borrows the benchmark’s one structural advantage.

E. Cost, and relation to prior work

Table VII sharpens the conclusion. SARIMAX is the most expensive model at 59.5 s and buys no aggregate advantage over LightGBM at 9.1 s; the GRU costs 12.0 s and the most memory (1,710 MB) for the worst accuracy; the baselines cost 0.1 s and win. On this dataset, at this scale, spending compute did not buy accuracy.

This agrees with Makridakis et al. [7]: on short series with strong regular seasonality, simple benchmarks are hard to beat. It looks like a conflict with the deep spatio-temporal results of [4], [5], but the two settings differ in ways that explain the gap. Those models train on thousands of cells and exploit explicit spatial structure, whereas ours train on seven cells with no spatial information; and where their comparisons use aggregate error against a non-seasonal ARIMA, ours uses per-cell MASE against a properly seasonal benchmark over a holiday period. Our result describes what is achievable in this regime, with few series, short history, strong seasonality and distribution shift at test time.

VII. CONCLUSION AND FUTURE WORK

This study built a memory-bounded pipeline for the 20.8 GB Milan telecom dataset, characterised its temporal, spatial and statistical structure, and compared three forecasting families on 7 cells under one protocol with leakage

rules enforced in code.

The main finding is that the 24-hour seasonal naive remains the best method overall (0.863), and we can say why: a holiday fortnight is mainly a level shift, and re-anchoring on a recent observation absorbs it without estimation, which gives the benchmark an error that barely grows with horizon. SARIMAX and LightGBM are indistinguishable in aggregate and are better read by horizon, with SARIMAX the strongest method in the study at one hour and LightGBM the stronger of the two at 24. The GRU does not justify its complexity on six weeks of fitting data. Model ranking varies systematically with horizon but not demonstrably with cell activity profile, and on that half of the research question the evidence does not support a positive claim.

Limitations. Only 7 cells and one activity were modelled. Two months contain few holidays, the validation week contains none, and the test period is unusually difficult, so absolute error levels should not be read as typical operating accuracy. Cluster labels interpret profile shape and are not ground-truth land use, and $k = 4$ was retained although the silhouette criterion preferred $k = 2$. Every configuration was trained once, with a single seed and a single seven-day validation window, so this study can support claims about large differences between model families but not about small differences between configurations within one. The LightGBM search varied only capacity and regularisation, leaving feature-group choice to the manual rounds. One final SARIMAX fit did not converge.

Future work. Four extensions follow from the specific failures above. First, give the models the benchmark’s advantage explicitly: forecast the residual from the last observed same-hour value, or re-estimate online, and test whether the long-horizon collapse disappears. Second, train on all 10 000 cells with cell-type embeddings, which would give the sequence model the training volume it clearly lacked. Third, add spatial neighbours’ lags to separate the value of spatial information from the effect of training scale in [4], [5]. Fourth, use a longer dataset covering several holiday periods. That is the only way to estimate holiday effects instead of just flagging them. Probabilistic forecasts would also suit capacity planning better, because operators care about the upper tail and not the conditional median.

Reproducibility. Complete source code, configuration, executed notebook, experiment log and instructions are in the GitHub repository [23], and the accompanying video presentation is at [24]. Every number in this report is generated from the result CSVs by `scripts/07_report_assets.py`.

USE OF AI TOOLS

An AI coding assistant (Cursor) was used to scaffold the repository, draft code for the ingestion pipeline, models and plotting, and to draft the structure of this report. All experimental decisions, including dataset choice, cell selection, model specifications, the tuning sequence, and the interpretation of results, were made and verified by the author, who ran the experiments and edited the text. The author takes full responsibility for the content.

REFERENCES

- [1] Telecom Italia, “Telecommunications – SMS, Call, Internet – MI,” 2015, v1. [Online]. Available: <https://doi.org/10.7910/DVN/EGZHFV>
- [2] G. Barlacchi, M. De Nadai, R. Larcher, A. Casella, C. Chitic, G. Torrisi, F. Antonelli, A. Vespignani, A. Pentland, and B. Lepri, “A multi-source dataset of urban life in the city of Milan and the Province of Trentino,” *Scientific Data*, vol. 2, p. 150055, 2015.
- [3] A. Furno, M. Fiore, R. Stanica, C. Ziemlicki, and Z. Smoreda, “A tale of ten cities: Characterizing signatures of mobile traffic in urban areas,” *IEEE Transactions on Mobile Computing*, vol. 16, no. 10, pp. 2682–2696, 2017.
- [4] J. Wang, J. Tang, Z. Xu, Y. Wang, G. Xue, X. Zhang, and D. Yang, “Spatiotemporal modeling and prediction in cellular networks: A big data enabled deep learning approach,” in *Proc. IEEE INFOCOM*, 2017, pp. 1–9.
- [5] C. Zhang, H. Zhang, D. Yuan, and M. Zhang, “Citywide cellular traffic prediction based on densely connected convolutional neural networks,” *IEEE Communications Letters*, vol. 22, no. 8, pp. 1656–1659, 2018.
- [6] C.-W. Huang, C.-T. Chiang, and Q. Li, “A study of deep learning networks on mobile traffic forecasting,” in *Proc. IEEE 28th Annual Int. Symp. on Personal, Indoor, and Mobile Radio Communications (PIMRC)*, 2017, pp. 1–6.
- [7] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, “Statistical and machine learning forecasting methods: Concerns and ways forward,” *PLoS ONE*, vol. 13, no. 3, p. e0194889, 2018.
- [8] T. Januschowski, Y. Wang, K. Torkkola, T. Erkkilä, H. Hasson, and J. Gasthaus, “Forecasting with trees,” *International Journal of Forecasting*, vol. 38, no. 4, pp. 1473–1481, 2022.
- [9] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: <https://otexts.com/fpp3/>
- [10] R. J. Hyndman and A. B. Koehler, “Another look at measures of forecast accuracy,” *International Journal of Forecasting*, vol. 22, no. 4, pp. 679–688, 2006.

- [11] F. X. Diebold and R. S. Mariano, "Comparing predictive accuracy," *Journal of Business & Economic Statistics*, vol. 13, no. 3, pp. 253–263, 1995.
- [12] D. Harvey, S. Leybourne, and P. Newbold, "Testing the equality of prediction mean squared errors," *International Journal of Forecasting*, vol. 13, no. 2, pp. 281–291, 1997.
- [13] D. A. Dickey and W. A. Fuller, "Distribution of the estimators for autoregressive time series with a unit root," *Journal of the American Statistical Association*, vol. 74, no. 366, pp. 427–431, 1979.
- [14] D. Kwiatkowski, P. C. B. Phillips, P. Schmidt, and Y. Shin, "Testing the null hypothesis of stationarity against the alternative of a unit root," *Journal of Econometrics*, vol. 54, no. 1–3, pp. 159–178, 1992.
- [15] R. B. Cleveland, W. S. Cleveland, J. E. McRae, and I. Terpenning, "STL: A seasonal-trend decomposition procedure based on loess," *Journal of Official Statistics*, vol. 6, no. 1, pp. 3–73, 1990.
- [16] K. Bandara, R. J. Hyndman, and C. Bergmeir, "MSTL: A seasonal-trend decomposition algorithm for time series with multiple seasonal patterns," 2021, arXiv:2107.13462. [Online]. Available: <https://arxiv.org/abs/2107.13462>
- [17] S. Seabold and J. Perktold, "statsmodels: Econometric and statistical modeling with Python," in *Proc. 9th Python in Science Conference*, 2010, pp. 92–96.
- [18] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 3146–3154.
- [19] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [20] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in *Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.
- [21] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*, 2019, pp. 8024–8035.
- [22] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. 25th ACM SIGKDD Int. Conf. on Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631.
- [23] S. Ihimbazwe, "Time-series-forecasting: source code for this study," 2026, gitHub repository. [Online]. Available: <https://github.com/Samkwizera/Time-Series-Forecasting>
- [24] —, "Forecasting mobile network traffic in milan: individual video presentation," 2026, demo video. [Online]. Available: <https://REPLACE-WITH-VIDEO-LINK>